

Lala AI — Mobile Application Architecture

Production Cross-Platform Architecture Blueprint for Android & iOS

TARGET PLATFORMS	DOCUMENT VERSION	DATE	STATUS
Android & iOS (Flutter)	1.0	September 2026	Proposed Architecture

Target Platforms: Android & iOS (Flutter Cross-Platform Engine) **Document Version:** 1.0 **Date:** September 2026 **Status:** Proposed Mobile Architecture

Executive Summary & Scope

This document defines the Mobile Application Architecture for **Lala AI**, a cross-platform content creator copilot built using Flutter. The architecture focuses strictly on the mobile application structure for Android and iOS, including presentation, state management, domain logic, security, social integrations, voice processing, and client API communication.

NOTE: This document strictly covers the Flutter mobile client application. Server infrastructure and backend microservices are excluded.

1. Flutter App Architecture

The Lala AI Flutter application follows a decoupled Clean Architecture combined with Feature-Driven Modularization. The application separates code into three core architectural layers (Presentation, Domain, and Data), supported by shared core modules and utilities.

Overall Flutter Architecture Diagram



Syntax error in text

mermaid version 10.9.8

Layer Separation & Responsibilities

Presentation Layer

- **UI Views & Widgets:** Renders Material/Cupertino widgets, handles layout responsiveness, and displays interactive components (e.g. ``MessageComposer``, ``VoiceSheet``, ``PlatformCard``).
- **State Management (Controllers):** Manages screen states in volatile memory, captures user inputs, triggers business operations, and updates the UI reactively.

Domain Layer

- **Entities:** Pure Dart objects representing core domain models (e.g. ``UserEntity``, ``ChatMessageEntity``, ``ChannelMetricsEntity``). Free from Flutter framework dependencies.
- **Repository Contracts (Interfaces):** Defines abstract contracts for fetching and mutating data via API services without framework coupling.

Data Layer

- **Data Transfer Objects (DTOs / Models):** Contains JSON serialization/deserialization logic (``fromMap``, ``toMap``, ``fromJson``).
- **Repository Implementations:** Implements domain interfaces, routing API requests through remote data sources.
- **Data Sources:** Wrappers around HTTP network clients and secure token storage services.

Recommended Folder & Project Structure

```

lib/
├─ main.dart                # Application Entry Point & Global Bootstrapping
├─ app/
│   ├── config/             # Environment Configurations & App Constants
│   │   ├── app_config.dart  # Base URLs, Timeouts, Feature Flags
│   │   └─ theme_config.dart # Gilroy Typography & Dynamic Palette
│   ├── routes/             # Named Navigation Routes & Auth Guards
│   │   ├── app_pages.dart   # Route Mapping & Transition Animations
│   │   └─ app_routes.dart   # Route String Constants
│   ├── modules/            # Self-Contained Feature Modules
│   │   ├── splash/          # Auth Status & Bootstrapping
│   │   ├── onboarding/      # Feature Walkthrough & Intro Flow
│   │   ├── authentication/   # Login, Register, Forgot Password
│   │   ├── main_container/   # Bottom Navigation Shell
│   │   ├── home/            # Creator Dashboard Overview
│   │   ├── chat/            # AI Copilot Chat & Voice Input
│   │   │   ├── bindings/
│   │   │   ├── controllers/
│   │   │   ├── views/
│   │   │   └─ widgets/      # MessageComposer, VoiceSheet, ChatBubble
│   │   ├── studio/          # AI Script & Content Studio
│   │   ├── calendar/        # Content Planning Calendar
│   │   ├── connect_accounts/ # YouTube & Instagram Channel Sync
│   │   ├── competitor/      # Competitor Tracking & Analysis
│   │   ├── category_insights/ # Niche Analytics & Audience Trends
│   │   ├── trending/        # Viral Trend Discovery
│   │   ├── saved/           # Saved Prompts & Bookmarked Scripts
│   │   ├── analytics/       # Deep Channel & Video Performance
│   │   ├── profile/         # Creator Profile & Subscription Tier
│   │   ├── settings/        # App Settings, Theme, Legal Policies
│   │   └─ faq/              # Creator Help & Knowledge Base
│   ├── data/               # App Data Layer
│   │   ├── models/         # Data Transfer Objects (DTOs)
│   │   │   ├── auth_model.dart
│   │   │   ├── chat_model.dart
│   │   │   ├── analytics_model.dart
│   │   │   └─ social_account_model.dart
│   │   ├── local/          # Token & Preference Wrappers
│   │   │   ├── app_storage_service.dart
│   │   │   └─ secure_storage_service.dart
│   │   └─ repositories/    # Concrete Repository Implementations
│   │       ├── auth_repository.dart
│   │       ├── chat_repository.dart
│   │       └─ social_repository.dart
│   ├── domain/             # Pure Business Rules & Interfaces
│   │   ├── entities/
│   │   │   ├── user_entity.dart
│   │   │   └─ chat_message_entity.dart
│   │   └─ repository_interfaces/
│   │       ├── i_auth_repository.dart
│   │       └─ i_chat_repository.dart
├─ networking/              # HTTP Client & Interceptors
└─ api_endpoints.dart        # Endpoint Registry

```

```
|   └─ api_response.dart      # Generic Response Envelope
|   └─ api_service.dart      # HTTP Client Engine & Interceptors
└─ utils/                    # Common Reusable Components & Utilities
    └─ app_toast.dart        # Toast & Notification System
    └─ common_widget.dart    # Reusable AppBars, Buttons, Cards, Shimmers
    └─ extensions.dart       # Padding & Formatting Helpers
    └─ theme/                # Color Palette & Typography Tokens
```

Reusable Components & Common Utilities

- **Common Widgets (`CW`)**: Standard app bars (`CW.commonAppBar`), dynamic action buttons (`CW.commonBtn`), input text fields, and elevated cards.
- **Color Constants (`CC`)**: Dynamic light and dark mode color system with primary teal accents, subtle borders, and semantic success/error indicators.
- **Text Styles (`TS`)**: Standardized typography hierarchy built on the Gilroy font family (`Gilroy-Regular`, `Gilroy-Medium`, `Gilroy-SemiBold`, `Gilroy-Bold`, `Gilroy-Heavy`).
- **Extensions**: Helper extensions for widget spacing (`16.height`, `12.width`), string capitalization, and date formatting.

2. State Management

Application state is managed using **GetX State Management** (`get: ^4.7.3`), providing reactive state updates (`Rx`), dependency injection (`Bindings`), and memory-efficient controller lifecycles.

State Categorization & Distribution



Syntax error in text
mermaid version 10.9.8

Feature State Specifications

Feature Module	State Living Location	State Details & Reactivity
Authentication	`AuthController` (Global `GetxService`)	Manages `isAuthenticated`, `currentUser`, login form validation, token storage status.
User Profile	`ProfileController`	Holds profile details, avatar URL, content niche, subscription tier (`Free`, `Pro`, `Enterprise`) fetched dynamically from API.
AI Chat	`ChatController`	Holds `RxList` in volatile memory. Fetches conversation history live from API on view initialization.
Social Connections	`ConnectAccountsController`	Manages `isYouTubeConnected`, `isInstagramConnected`, platform handles, and connection statuses fetched live via API.
Notifications	`SettingsController` / `NotificationService`	Controls push notification permission flags and creator tip alerts.
Settings / Preferences	`ThemeService` (Global `GetxService`)	Persists dark/light theme mode preference and system language locale using `SharedPreferences`.

UI View States (Loading, Success, Error, Empty)

All async operations use a standardized view state model:

```
enum ViewStatus { initial, loading, success, error, empty }
class ViewState {
  final ViewStatus status;
  final T? data;
  final String? message;

  ViewState.loading() : status = ViewStatus.loading, data = null, message = null;
  ViewState.success(this.data) : status = ViewStatus.success, message = null;
  ViewState.error(this.message) : status = ViewStatus.error, data = null;
  ViewState.empty() : status = ViewStatus.empty, data = null, message = null;
}
```

- **Loading State:** Displays shimmers or localized activity indicators while keeping the navigation header interactive.
- **Success State:** Renders data widgets.
- **Error State:** Renders error illustrations with clear retry actions (`CW.commonBtn(title: "Retry")`).

- **Empty State:** Renders friendly contextual placeholders when API responses return empty arrays (e.g. `"No saved scripts yet"`).

3. Navigation Architecture

Navigation uses named GetX routes (``AppPages``) configured with transitions and route guards.

Navigation Hierarchy & Flow



Syntax error in text
mermaid version 10.9.8

Route Guards

- **Authentication Guard (``AuthGuard``):** Protects routes requiring authentication (``/main_container``, ``/chats``, ``/studio``, ``/connect_accounts``). Unauthenticated attempts are redirected to ``/authentication``.

4. API Integration

The application uses REST APIs for retrieving and submitting application data. Based on the current requirements, no offline database or local chat cache is included in the proposed architecture.

Mobile-Side API Data Flow



Syntax error in text
mermaid version 10.9.8

Component Placement Rules

- **API Models / DTOs (``lib/app/data/models``):** Parses raw JSON payloads into strongly-typed Dart objects (``fromJson``/``toJson``). Includes null safety fallback logic.
- **API Services (``lib/networking``):** Low-level HTTP engine managing timeouts (15s), Bearer token injection, logging, and standard HTTP exception parsing.

- **Repositories** (``lib/app/data/repositories``): Executes HTTP API requests and maps DTOs directly to pure Domain Entities.
- **Domain Entities** (``lib/app/domain/entities``): Frame-independent business models consumed by GetX Controllers in memory.

5. Local Storage & Database Architecture

To keep the mobile application lightweight and secure, the current architecture does not require an offline database or local chat cache. Chat history, conversations, content scripts, and creator channel analytics are retrieved through application APIs when required.

Storage Strategy & Component Classification

IMPORTANT: Storage Policy: Chat conversations, messages, and user content are not persisted in a local database in the current design. Active chat/content data is held in application memory while the relevant feature is in use.

Data Category	Target Persistence	Storage Mechanism	Eviction / Scope Policy
Auth Access & Refresh Tokens	Hardware Encrypted Storage	<code>`FlutterSecureStorage`</code>	Cleared on Manual Logout or Token Revocation
User Session Metadata	Hardware Encrypted Storage	<code>`FlutterSecureStorage`</code>	Stores <code>`userId`</code> and <code>`subscriptionTier`</code>
App Theme & UI Preferences	Key-Value Preference Store	<code>`SharedPreferences`</code>	Permanent (User Configured - Dark/Light mode, Locale)
Chat History & Messages	No Local Storage	In-Memory (<code>`ChatController`</code> RAM)	Disposed when screen is closed or user logs out
Content Scripts & Drafts	No Local Storage	In-Memory (<code>`StudioController`</code> RAM)	Disposed when screen is closed or user logs out
Channel Analytics & Trends	No Local Storage	In-Memory (<code>`AnalyticsController`</code> RAM)	Fetches live via API

Local Database Decision

Local Database: TBD / Not currently required

The current mobile architecture does not require a dedicated local relational or NoSQL database. Local persistence is limited to secure session information and lightweight application preferences.

If future requirements introduce offline chat history, drafts, or offline-first functionality, a local database can be introduced after confirming the data model and offline requirements. The database technology should be selected at that stage rather than being assumed in the current architecture.

6. Authentication & Security

Security enforces a Zero-Trust client architecture to protect creator channel data and API tokens.

Authentication & Token Refresh Flow



Syntax error in text
mermaid version 10.9.8

Security Measures

1. **Secure Token Storage:** JWT access and refresh tokens are encrypted using `EncryptedSharedPreferences` on Android and iOS `Keychain` via `FlutterSecureStorage`. 2. **Session Cleanup on Logout:** Executing logout clears all secure tokens from `FlutterSecureStorage`, flushes UI preferences in `SharedPreferences`, purges all in-memory controller states, and redirects to the login screen. 3. **Secure API Transport:** Enforces HTTPS over TLS 1.3 for all REST interactions. Sensitive credentials (OAuth secrets, client secrets) are **never** stored inside the Flutter codebase. 4. **Certificate Pinning:** Marked as **TBD / To Be Confirmed** subject to API domain deployment.

7. Instagram & YouTube Integration

Social account connection is designed around an authorization flow initiated from the mobile application.

Mobile-Side Webview OAuth Integration Sequence



Syntax error in text
mermaid version 10.9.8

8. AI / Voice Architecture

Lala AI supports dual-modal interaction: typed text input and continuous voice speech recognition.

Mobile AI & Voice Data Flow



Syntax error in text
mermaid version 10.9.8

Multilingual Voice Strategy

Native Speech-to-Text Engine (`speech\_to\_text: ^7.4.0`)

- Utilizes native OS speech recognition frameworks (Apple Speech Framework on iOS and Google On-Device Speech Recognition on Android).
- Automatically detects user device locale (e.g., `en\_US`, `hi\_IN`, `mr\_IN` for English, Hindi, Marathi).

Unified Language Handling

- Transcribed text (regardless of script e.g. English, Devanagari) is sent directly to the API service. The language processing model handles multi-lingual intent understanding without requiring separate per-language code paths in Flutter.

Text-To-Speech (TTS)

- Marked as **TBD / To Be Confirmed** pending client approval on whether responses should be read aloud natively via `flutter\_tts`.

9. Android & iOS

Platform-specific integrations are configured via native project files.



Syntax error in text
mermaid version 10.9.8

Platform Configuration Specifications

Android Setup (`android/`)

- **Minimum SDK:** 24 (Android 7.0) | **Target SDK:** 34 (Android 14)
- **Permissions (`AndroidManifest.xml`):**
 - ``
 - ``
 - ``

iOS Setup (`ios/`)

- **Minimum Deployment Target:** iOS 14.0
 - **Privacy Keys (`Info.plist`):**
 - `NSMicrophoneUsageDescription`: `"Lala AI needs microphone access to record voice prompts."`
 - `NSSpeechRecognitionUsageDescription`: `"Lala AI uses speech recognition to convert your voice into script prompts."`
-

10. Technology Matrix & Selection Rationale

Architectural Component	Confirmed Technology	Target Platform	Reason for Selection
Framework Engine	Flutter (SDK ^3.9.2)	Android + iOS	Cross-platform UI performance (60fps), single Dart codebase, fast iteration cycle.
State Management	GetX ('get: ^4.7.3')	Android + iOS	Decoupled routing & in-memory reactive state without 'BuildContext', low boilerplate overhead.
Encrypted Token Storage	'flutter_secure_storage'	Android + iOS	Hardware-backed Keychain (iOS) and KeyStore / EncryptedSharedPreferences (Android) for JWT security.
KeyValue Preferences	'shared_preferences: ^2.5.5'	Android + iOS	Lightweight storage strictly for user theme preferences, onboarding flags, and app locale.
In-App OAuth Webview	'webview_flutter: ^4.10.0'	Android + iOS	Secure in-app browser with navigation delegates for capturing OAuth callback parameters without external app switches.
Networking Client	'http: ^1.3.0'	Android + iOS	Custom HTTP client, header management, Bearer token injection, and chunked SSE stream support.
Voice / Speech Engine	'speech_to_text: ^7.4.0'	Android + iOS	Native OS speech framework binding (Apple Speech / Google On-Device Speech) with locale auto-detection.

11. TBD / To Be Confirmed

Items requiring client confirmation or final technical validation are tracked below:

#	Feature / Component	Status	Recommendation / Options
1	Certificate Pinning	TBD	Recommend pinning SSL certificates once API domains are finalized.
2	Text-To-Speech (TTS)	TBD	Confirm if AI script responses should be read aloud natively via `flutter_tts`.
3	Push Notification Provider	TBD	Recommend Firebase Cloud Messaging (FCM) for channel analytics milestone alerts.